

AI Infrastructure Intelligence Platform

Executive Presentation & Technical Demo Guide | Enterprise Management System

Project Overview	An enterprise-grade platform for multi-server monitoring, automated web project discovery (WHM/SSH), AI predictive risk scoring, and cloud management.
Deployment Host	Railway Cloud (Unified Multi-Stage Docker Container)
Database	PostgreSQL (Supabase Cloud Pooler with SQLAlchemy ORM)
Test Suite Verification	47 / 47 Automated Unit & Integration Tests Passing (100% Clean)

1. Technology Stack Breakdown (What We Used)

Layer	Technology Used	Purpose / Role in Application
Frontend UI	React 18, JavaScript, Axios, React Router, Recharts	Interactive single-page application (SPA). Recharts renders live server load & risk visual charts. Axios handles API calls.
Backend API	Python 3.11, FastAPI, Uvicorn, Pydantic	Asynchronous high-performance REST API endpoints, request validation, data serialization, and background task orchestration.
Database	PostgreSQL, Supabase, SQLAlchemy 2.0 ORM	Persistent relational data storage with connection pooling (pool_size=10, max_overflow=20, pool_pre_ping=True).
Container & Hosting	Docker, Railway Cloud, Alpine Linux	Multi-stage Dockerfile compiles React frontend assets and mounts Uvicorn backend runner in one single Railway container.
Machine Learning	Scikit-Learn, XGBoost, MLflow 2.14, NumPy, Pandas	RandomForestRegressor model predicting server risk scores (0-100). MLflow tracks experiment metrics and model drift.

2. Security & Authentication Architecture (What for Login?)

- OAuth2 with Password Flow & JWT Tokens:** Login requests emit a signed JSON Web Token (JWT) using python-jose with HS256 signature algorithm and configurable token expiration.
- Password Security:** Uses bcrypt salted password hashing (via passlib). Raw passwords are never saved in database.
- Credential Fernet Encryption:** SSH passwords, SSH private keys, and WHM API root tokens are encrypted at rest in PostgreSQL using cryptography.fernet symmetric encryption.
- Rate Limiting Guard:** Integrated SlowAPI limiter on /auth/login to prevent brute-force attacks.
- Role-Based Access Control (RBAC):** Supports 3 distinct user permission roles: admin (full control), devops (manage & scan), and viewer (read-only monitoring).

3. Machine Learning & Predictive Risk Engine (What in ML?)

- **RandomForest Regression Model:** risk_engine.py runs predictions using a trained RandomForestRegressor model (risk_model.pkl) evaluating 5 core server metrics:
 1. CPU Usage (%) | 2. RAM Consumption (%) | 3. Storage Disk Usage (%) | 4. Server Uptime (Days) | 5. 24-Hour Critical Log Error Count
- **MLflow Experiment Management:** Tracks training runs, logs R^2 score, Mean Squared Error (MSE), and Mean Absolute Error (MAE) into MLflow 2.14 tracking database.
- **Model Drift Monitoring:** Automatic pipeline evaluator detects model degradation and sets drift_detected: true if test set R^2 drops below 0.70 threshold.
- **Deterministic Fallback Guard:** If the ML model is being retrained, the platform seamlessly uses deterministic industry-standard threshold scoring so predictions never fail.

4. Real-Time Discovery & Auto-Refresh Infrastructure

- **Dual WHM / SSH Live Scanning:** WHM mode queries /json-api/listaccts for live cPanel accounts; SSH mode executes native Linux commands (df -h, free -m, lscpu) over Paramiko.
- **WAF Security Resilience:** Session persistence and browser header matching allow clean API access through Imunify360 and cPHulk protection barriers.
- **Frontend 30-Second Auto-Polling:** React DataContext.jsx automatically re-fetches dashboard states, server health, and discovery records every 30 seconds.
- **Backend Hourly APScheduler:** Background scheduler (hourly_sync_job) executes hourly background re-scans of all registered servers.

5. Higher-Up Demo Q&A; Cheat Sheet (Executive Q&A;)

Q1: What technologies were used to build this platform?

We built the frontend using React 18 with Recharts for dynamic visual dashboards. The backend is powered by Python 3.11 with FastAPI for high-performance asynchronous API endpoints. The database is PostgreSQL hosted on Supabase, managed via SQLAlchemy 2.0 ORM, and deployed on Railway using a multi-stage Docker container.

Q2: How does authentication and login security work?

Login uses OAuth2 with JSON Web Tokens (JWT) signed via python-jose (HS256 algorithm). Passwords are encrypted using bcrypt salted hashing. Sensitive credentials like SSH keys and WHM tokens are encrypted at rest using Fernet symmetric encryption. We also enforce Rate Limiting (SlowAPI) to prevent brute-force attacks.

Q3: How does the Machine Learning engine predict server risk?

We use a Scikit-Learn RandomForestRegressor model trained on 5 real-time system metrics: CPU usage, Memory consumption, Disk space, Server Uptime, and Critical Log Error counts. It predicts a Server Risk Score from 0 to 100. We also track hyperparameter experiments in MLflow 2.14 and monitor for Model Drift.

Q4: Is the server and project discovery data live or simulated?

It is 100% live real-time telemetry. There are zero fallback lists or hardcoded datasets in the codebase. WHM mode queries WHM REST API (/json-api/listaccts) over HTTPS, while SSH mode connects via Paramiko to execute native Linux kernel commands.

Q5: How does the system handle security protections like WAFs or Imunify360?

Our WHM client uses persistent HTTP sessions with browser User-Agent headers, TLS verification controls, and explicit HTTP status code handling (403/401/429), capturing diagnostic error messages cleanly without crashing database transactions.

Q6: Does the application refresh data automatically?

Yes. On the frontend, React Context automatically polls and updates dashboard metrics every 30 seconds. On the backend, APScheduler runs an automated hourly sync job that rescans all servers, updates metrics, detects duplicate/inactive projects, and refreshes AI insights.

Q7: How does duplicate project detection work?

Our duplicate detection engine uses a multi-signal approach: matching exact domain names, Git remote repository URLs, and fuzzy string similarity using Python's SequenceMatcher with an 85% similarity threshold.

Q8: How are abandoned or inactive projects identified?

The inactive project engine cross-references cPanel account suspension flags, DNS/SSL resolution checks, and filesystem modification timestamps (>1000 days old) to generate actionable recommendations like 'archive' or 'delete' to free up disk space.

Q9: What database pooling strategy is used for production stability?

We configured SQLAlchemy connection pooling with pool_size=10, max_overflow=20, pool_timeout=30 seconds, and pool_pre_ping=True to automatically verify and refresh stale database connections.

Q10: Is the application tested and verified?

Yes. We maintain a full test suite with 47 automated unit and integration tests passing cleanly in pytest, verifying API authentication, server CRUD operations, discovery algorithms, and risk calculations.
